

ĐỀ THI MẪU – LẬP TRÌNH NÂNG CAO

(Chỉ dùng để tham khảo cấu trúc và khối lượng đề)

Lưu ý: Đây là đề mẫu phục vụ ôn tập. Nội dung và tình huống khác với đề thi chính thức.

BÀI 1. TRẮC NGHIỆM (minh hoạ) - 5 điểm

ĐOẠN CODE 1

```
class Employee {
    protected String name;
    private double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public double income() {
        return salary;
    }

    public String role() {
        return "Employee";
    }

    public static String policy() {
        return "standard";
    }
}

class Manager extends Employee {
    private double bonus;

    Manager(String name, double salary, double bonus) {
        super(name, salary);
        this.bonus = bonus;
    }

    @Override
    public double income() {
        return super.income() + bonus;
    }
}
```

```

@Override
public String role() {
    return "Manager";
}

public static String policy() {
    return "management";
}
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Manager("Lan", 1000, 300);

        System.out.println(e.role());
        System.out.println(e.income());
        System.out.println(e.policy());
        System.out.println(((Manager) e).policy());
    }
}

```

CÂU HỎI CHO ĐOẠN CODE 1

1. Output của chương trình là gì?
2. Vì sao lời gọi `e.role()` cho kết quả khác với `e.policy()`?
3. Nếu bỏ annotation `@Override` khỏi `income()` thì điều gì xảy ra?
4. Nếu thêm:

```

Employee another = new Employee("A", 10);
System.out.println(((Manager) another).income());

```

thì điều gì xảy ra? 5. Nếu đổi:

```
public double income()
```

trong Manager thành:

```
private double income()
```

thì điều gì xảy ra? 6. Thuộc tính salary có thể được truy cập trực tiếp trong lớp Manager hay không? Vì sao?

ĐOẠN CODE 2

```
import java.util.*;

interface Printable {
    String print();
}

class Report implements Printable {
    private final String title;

    Report(String title) {
        this.title = title;
    }

    public String print() {
        return "Report:" + title;
    }

    public String toString() {
        return print();
    }
}

class Storage<T extends Printable> {
    private final List<T> items = new ArrayList<>();

    void save(T item) {
        items.add(item);
    }

    void printAll() {
        for (T item : items) {
            System.out.println(item.print());
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Storage<Report> storage = new Storage<>();
        storage.save(new Report("Weekly"));
        storage.save(new Report("Monthly"));
        storage.printAll();
    }
}
```

CÂU HỎI CHO ĐOẠN CODE 2

7. Output của chương trình là gì?

8. Ý nghĩa của khai báo:

```
class Storage<T extends Printable>
```

là gì?

9. Dòng mã nào sau đây gây lỗi biên dịch?

- A. `Storage<Report> s = new Storage<>();`
- B. `Storage<Printable> s = new Storage<>();`
- C. `Storage<String> s = new Storage<>();`
- D. `Report r = new Report("A");`

10. Vì sao phương thức `print()` trong lớp `Report` phải là `public`?

11. Nếu lớp `Report` không override `toString()` thì điều gì thay đổi trong output hiện tại?

12. Thiết kế `Storage<T>` giúp ích gì so với việc dùng trực tiếp `List<Object>`?

13. `Printable` phù hợp nhất với vai trò nào trong thiết kế hướng đối tượng?

ĐOẠN CODE 3

```
import java.util.concurrent.*;

class DownloadTask implements Callable<String> {
    private final String file;

    DownloadTask(String file) {
        this.file = file;
    }

    @Override
    public String call() throws Exception {
        Thread.sleep(100);
        return "Done:" + file;
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(2);

        Future<String> f1 = pool.submit(new DownloadTask("A"));
        Future<String> f2 = pool.submit(new DownloadTask("B"));

        System.out.println(f1.get());
        System.out.println(f2.get());

        pool.shutdown();
    }
}
```

CÂU HỎI CHO ĐOẠN CODE 3

14. Vai trò của Callable<String> khác gì với Runnable?
15. Output của chương trình là gì?
16. Vì sao Future.get() có thể làm chương trình tạm dừng?
17. Điều gì có thể xảy ra nếu bỏ pool.shutdown()?
18. Nếu thay Callable<String> bằng Runnable thì đoạn nào trong chương trình cần thay đổi?
19. Điều gì xảy ra nếu gọi:
f1.get();
f1.get();
20. ExecutorService giúp giải quyết vấn đề gì so với việc tự tạo nhiều Thread bằng tay?

BÀI 2 – 2.5 điểm

Một hệ thống quản lý thư viện số có người dùng gồm độc giả và thủ thư. Tài nguyên thư viện gồm sách điện tử, video học tập và tài liệu PDF. Một số tài nguyên có thể tải xuống, một số chỉ được xem trực tuyến. Người dùng có thể mượn tài nguyên. Hệ thống cần ghi nhận lịch sử mượn/trả. Một số tài nguyên có giới hạn số lượt truy cập đồng thời.

Yêu cầu:

1. Xác định các lớp chính và quan hệ giữa các lớp.
2. Sử dụng abstract class và interface hợp lý.
3. Vẽ biểu đồ lớp mức khái quát.
4. Nếu hệ thống bổ sung audiobook và cơ chế thuê tài nguyên theo thời hạn, hãy giải thích thiết kế cần mở rộng như thế nào để hạn chế sửa mã nguồn cũ.

BÀI 3 – 1 điểm

Cho phương thức:

```
boolean canBorrow(int borrowedBooks, boolean hasOverdue, int membershipLevel)
```

Quy tắc:

- Người dùng được mượn sách nếu:
 - `borrowedBooks < 5`
 - `hasOverdue == false`
 - `membershipLevel >= 1`
- Nếu `borrowedBooks < 0` hoặc `membershipLevel < 0` thì dữ liệu không hợp lệ.

Yêu cầu: Sinh bộ test theo kỹ thuật phân tích giá trị biên.

BÀI 4 – 1.5 điểm

Cho đoạn mã sau:

```
class Logger {
    void log(String level, String message) {
        if (level.equals("INFO")) {
            System.out.println("INFO: " + message);
        } else if (level.equals("ERROR")) {
            System.out.println("ERROR: " + message);
        } else if (level.equals("DEBUG")) {
            System.out.println("DEBUG: " + message);
        }
    }
}
```

Yêu cầu:

1. Chỉ ra các vấn đề thiết kế hoặc code smell trong đoạn mã.
2. Đề xuất cách cải thiện thiết kế và giải thích vì sao thiết kế mới dễ mở rộng hơn khi bổ sung loại log mới.